

EMBER — Full Project Brief

Hackathon: CS Girllies "Technology for Wellness"

Deadline: Aug 16, 2026, 11:00pm GMT+1

Track: Wellness (main) + Best Use of AI (bonus)

1. THE PITCH (say this exactly, don't reword into something generic)

Most people who try to track how they're feeling quit within a week — a blank journal page or a mood-tracker streak feels like homework, with nothing to show for it afterward.

Ember flips that. Every time you type what's actually going on — good or hard — it doesn't just respond, it **plants something in a persistent visual world that's uniquely yours**. Hard entries take root in one part of the landscape; good entries grow in another. Over days, you're not looking at old text or a streak number — you're looking at a small world you built, one honest entry at a time, that visibly holds both the weight and the light of what you've actually been through.

Each entry also returns:

1. One sharp line showing you were heard (specific to what you wrote, not generic)
2. One small, doable action for right now (not vague advice)
3. A new element planted in your world, in the correct zone

A **Guardrail Agent checks every entry FIRST**, before anything else runs. If it detects real crisis language, nothing gets planted, nothing gets clever — the user is shown real resources (988, Crisis Text Line) immediately and plainly. This overrides everything else. Non-negotiable, build this first and test it thoroughly.

Target audience: Students / young adults who abandon journaling or mood apps fast because blank pages + streak guilt don't hold attention. This reframes tracking-how-you-feel as *tending a place*, not doing homework.

Why this wins on the judging criteria (5 categories, evenly weighted per rules):

- *Wellness Impact:* answers "would you use this weekly" — the world only means something if you return to it
- *Creativity & Innovation:* inversion of generic mood-chatbots; nobody else is doing "accumulated visual world," verified against actual past winners of this same host (Sweet Treatinator, Compliment Generator, MoodMorph — all single-shot, none accumulate)
- *Technical Craft:* two-agent orchestration (Guardrail + Reflector) is real, working backend — rules explicitly say backend isn't required but scores higher if present
- *Design & UX:* the world canvas IS the UX centerpiece — build it with real craft, not a placeholder
- *Community & Accessibility:* frame the "always-heard-first, never-judged" mechanic honestly in the pitch as accessible emotional expression across backgrounds

Important honesty constraint: This is NOT therapy and should never be pitched as diagnosing/treating anything. It's a small daily practice of naming what's true. Say this explicitly in the demo — judges respect honest scoping over overclaiming.

2. ARCHITECTURE (already partially built — see /backend folder in this zip)

Backend (FastAPI, Python)

Two-agent pipeline, in this exact order:

1. **Guardrail Agent** (`prompts.py` → `GUARDRAIL_SYSTEM_PROMPT`) — runs on EVERY entry first. Classifies input as "crisis" or "safe" via JSON response. Fails safe: if the response can't be parsed, default to "crisis" (never default to safe on parse failure).
 - If `crisis`: return `CRISIS_RESPONSE_TEXT` immediately (988, Crisis Text Line, IASP link). Do NOT run the Reflector. Do NOT store/plant anything.
2. **Reflector Agent** — needs a rewrite from the current version (current version only does mirror-only reflection; NEEDS to be extended to also output):
 - `zone`: "positive" or "negative" (sentiment classification)
 - `reflection_line`: one sharp sentence, user's own words restructured, no advice/no affirmation/no AI-as-authority language
 - `micro_action`: one small, concrete, doable-in-10-minutes action tied to what they wrote (this part is NEW, not yet in `prompts.py` — add it)

Endpoints needed:

- `POST /entry` — takes `{text: string}`, runs Guardrail → Reflector, stores result if safe, returns `{mode, zone, reflection_line, micro_action, element_seed}`
- `GET /world` — returns all stored entries for the session (for rendering the full accumulated world on load)
- `GET /health` — already built

Persistence: SQLite. Table `entries`: `id, timestamp, raw_text, zone, reflection_line, micro_action, position_seed`. No auth needed — single local session ID is fine for hackathon scope, say so honestly in the README if asked.

Already built (in this zip, reuse don't rewrite):

- `backend/main.py` — FastAPI app, Guardrail+Reflector call pattern, CORS setup, Anthropic client setup
- `backend/prompts.py` — Guardrail prompt (keep as-is, it's tested), Reflector prompt (extend per above), crisis response text (keep as-is)

Frontend (Next.js + Tailwind)

Views:

1. Input box — single text field, submit button, minimal
2. Response card — shows `reflection_line` + `micro_action` after submit (typography-forward, large serif/humanist sans for the reflection line — this is the emotional centerpiece)
3. **World canvas** — the core visual piece:
 - Canvas/SVG scene, split by a **soft horizontal or diagonal gradient** (not a hard border) — one side cooler/darker for negative-zone entries, one side warmer/lighter for positive-zone entries

- Each stored entry = one positioned SVG element (simple organic shapes — soft blobs, circles, particles — NOT literal tree/rock illustrations, too slow to build well)
- Position determined deterministically from `position_seed` so layout doesn't reshuffle on reload
- New element: gentle glow/settle-in animation on creation (1-2 sec), then goes still — this animation IS the "fun" payoff, don't skip it
- Fetch `/world` on page load to render existing entries; append new element after each successful `/entry` call, no full page reload

Visual direction:

- Dark-mode base, warm accent colors (not clinical blue/white)
- Typography carries emotional weight — the reflection line should be the visual focus of its card, large, on its own line
- Motion only on entry-creation, otherwise calm/still (avoid busy constant animation)

Deploy

- Backend → Render
- Frontend → Vercel
- Standard stack for this dev, no new tooling needed

3. BUILD ORDER (in priority sequence — if time runs out, stop after step 5, that's still a complete demo)

1. Guardrail Agent — test thoroughly with real crisis-language inputs, confirm it always fires correctly, fails safe
2. Reflector Agent rewrite — add zone + micro_action to existing mirror logic
3. Backend `/entry` and `/world` endpoints + SQLite wiring
4. Frontend input → response card flow (functional, minimal styling)
5. World canvas — gradient field + deterministic element placement + entry animation
6. Deploy both, test end-to-end on live URLs
7. Demo video script (see below)
8. Written project description (MUST be written by hand, not AI — hackathon rules penalize AI-generated docs)

4. DEMO VIDEO STRUCTURE (2-3 min, don't exceed 5)

1. **Open cold** with a real entry typed live — a specific, relatable "bad day" sentence, not a generic example
2. Show the reflection + micro-action + element planting into the world — this is the "wow" beat
3. **Contrast moment:** type a second, lighter entry — show it planting into the OTHER zone, world visibly growing on both sides
4. **Guardrail demo:** type a crisis-flagged input, show the hard override to real resources — say explicitly why this matters ("most mood apps don't check for this")
5. Zoom out on the accumulated world after a few entries — this is the emotional payoff shot

6. Close with the "why" in one sentence — reference the real problem (people abandon journaling because it feels like homework)
-

5. THINGS TO NOT DO (learned the hard way in planning this)

- Do NOT add unrelated technical complexity (no WebGPU, no zk-SNARKs, no edge ML, no local LLM inference) — verified against actual past winners of this same host's hackathon, none used anything like this, all won with simple, well-executed single-concept builds
- Do NOT let the AI write the project description — hand-write it, hackathon explicitly penalizes AI-generated docs
- Do NOT skip the Guardrail Agent under time pressure — it is the single most important piece given the Wellness track's explicit rule about mood-bot guardrails
- Do NOT let the world visual become a literal illustrated garden with detailed tree/rock assets — scope creep risk, use simple organic shapes instead